

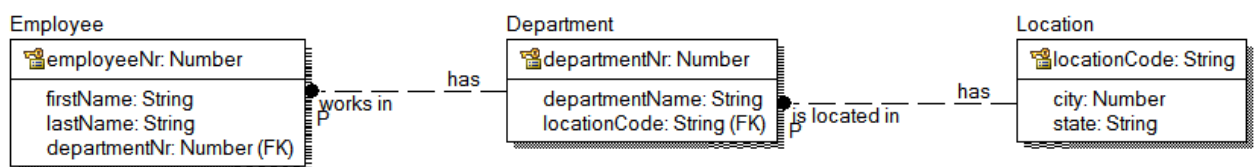
Abgabetermine: 16. April 2026, 12:00 Uhr
Abgabe elektronisch im E-Learning

☒ DEM2G1 Dr. Pitzer	Name <u>Klemens Danner</u>	Aufwand in h <u>13</u>
☐ DEM2G2 Dr. Pitzer		
☐ DEM2G3 Dr. Niklas	Punkte _____	Kurzzeichen Tutor _____

Hinweise und Richtlinien:

- Übungsausarbeitungen müssen den angegebenen Formatierungsrichtlinien entsprechen – Nichtbeachtung dieser Formatierungsrichtlinien führt zu Punkteabzug.
- Zusätzlich zu den allgemeinen Formatierungsrichtlinien sind für diese Übungsausarbeitung folgende zusätzlichen Richtlinien zu beachten:
 - Verwenden Sie zur Modellierung ERwin 7.2 oder 15.2. Für die elektronische Abgabe bitte auf die gute Lesbarkeit der Diagramm im PDF achten.
 - Vergessen Sie nicht Beziehungstypen, Primärschlüssel, Kardinalitäten, Funktionalitäten sowie die Optionalität von Attributen auszuarbeiten!
 - Achten Sie auf eine einheitliche Benennung der Entitäten und Beziehungen
 - Entweder immer Englisch *oder* immer Deutsch.
 - einheitliche Schreibweise - es empfiehlt sich der Einsatz von in „CamelCase“ geschriebenen Namen, z.B. GutachterTeam
 - Treffen Sie, falls notwendig, sinnvolle Annahmen und dokumentieren Sie diese nachvollziehbar in ihrer Lösung!
 - Bei Gruppenarbeit geben Sie die Namen aller Gruppenmitglieder an. Es wird nur eine Abgabe korrigiert.

Ziel dieser Übung ist es, Ausschnitte der Realität mit Hilfe von Entity-Relationship-Diagrammen werkzeugunterstützt zu modellieren und zu verbessern und daraus das relationale Modell abzuleiten. Erstellen Sie für die folgenden Aufgabenstellungen ER-Modelle und die dazugehörigen Relationenmodelle! Kennzeichnen Sie im Relationenmodell die jeweiligen Primärschlüssel und Fremdschlüssel sowie Kombinationen davon und geben Sie Datentypen an!



Location: {[locationCode:String, city:Number, state:String]}

Department: {[departmentNr:Number, departmentName:String, locationCode:String]}

Employee: {[employeeNr:Number, firstName:String, lastName:String, departmentNr:Number]}

1. Laborverwaltung

(4 Punkte)

Sie haben die Modellierung eines Laborverwaltungssystems gegeben. Jedes Labor hat einen zuständige*en Admin, die Studenten können die Labors nutzen, wobei in einem Labor mehrere Studenten gleichzeitig arbeiten können. Es soll gespeichert werden, wann ein bestimmter Student in einem Labor war (Nutzungsbeginn) und bis wann er in dem Labor war (Nutzungsende).

Das gegebene ER-Diagramm wurde bereits in ein Relationenmodell überführt. Verbessern Sie dieses Relationenmodell und korrigieren Sie Fehler, damit es dem ER-Modell entspricht. Achten Sie darauf, dass keine NULL-Werte entstehen und markieren Sie die jeweiligen Primärschlüssel und Fremdschlüssel sowie Kombinationen davon.

Person: {[PersonNr:Number, Name:String, Geburtsdatum:Date]}

Person_is_a: {[PersonNr:Number, AdministratorFlag:String, StudentFlag:String]}

Administrator: {[Gehalt:Number, Arbeitsbeginn:Date, LaborId:Number]}

Student: {[Studiengang:String, Semester:Number, LaborId:Number, Nutzungsbeginn:Date]}

Labor: {[LaborId:Number, Sitzplätze:Number, Typ:String, HardwareId:Number]}

Hardware: {[HardwareId:Number, Bezeichnung:String, Kaufpreis:Number, Kaufdatum:Date]}

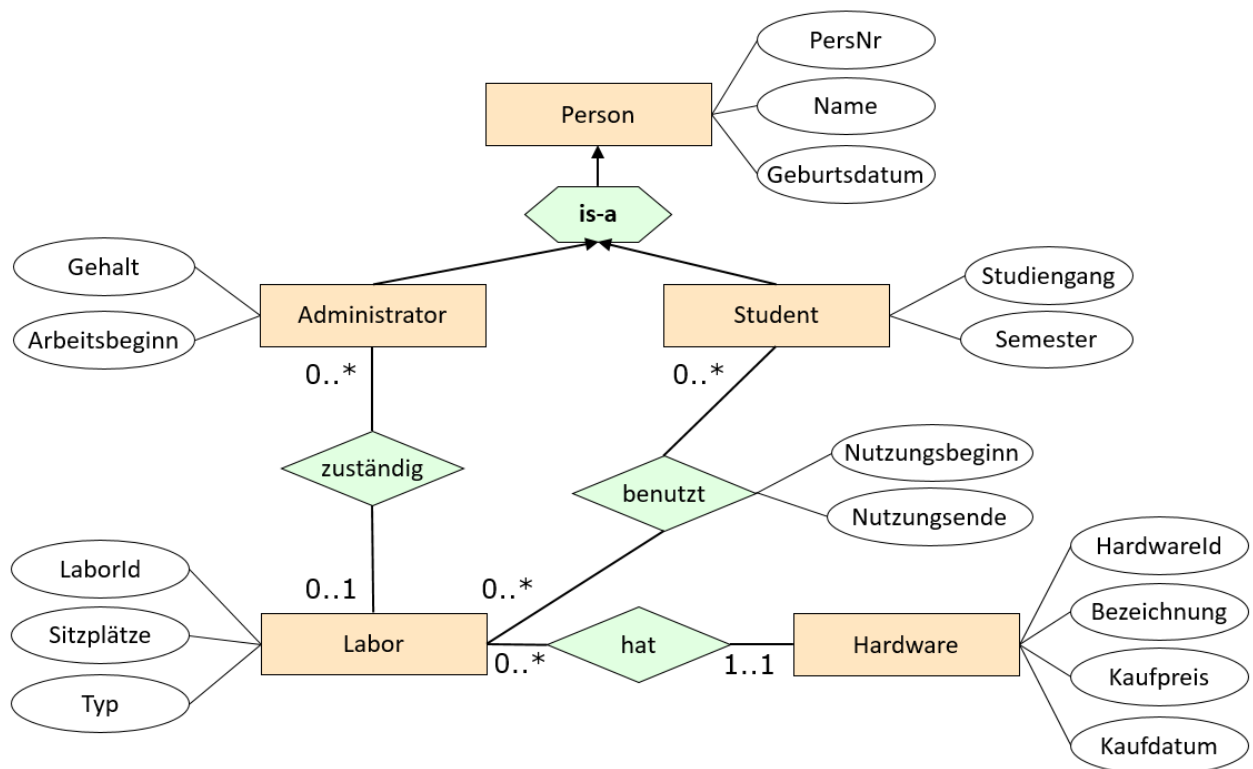


Abbildung 1: Laborverwaltung

2. Relationales Modell

(6 Punkte)

a) Wandeln Sie das folgende Entity-Relationship-Diagramm (= konzeptuelles Schema) in Abbildung 2 in ein relationales Modell (= logisches Schema) um. Verwenden Sie möglichst wenige Relationen und beachten Sie, dass die Datenbank **keine NULL-Werte** erlaubt (5,5 Punkte).

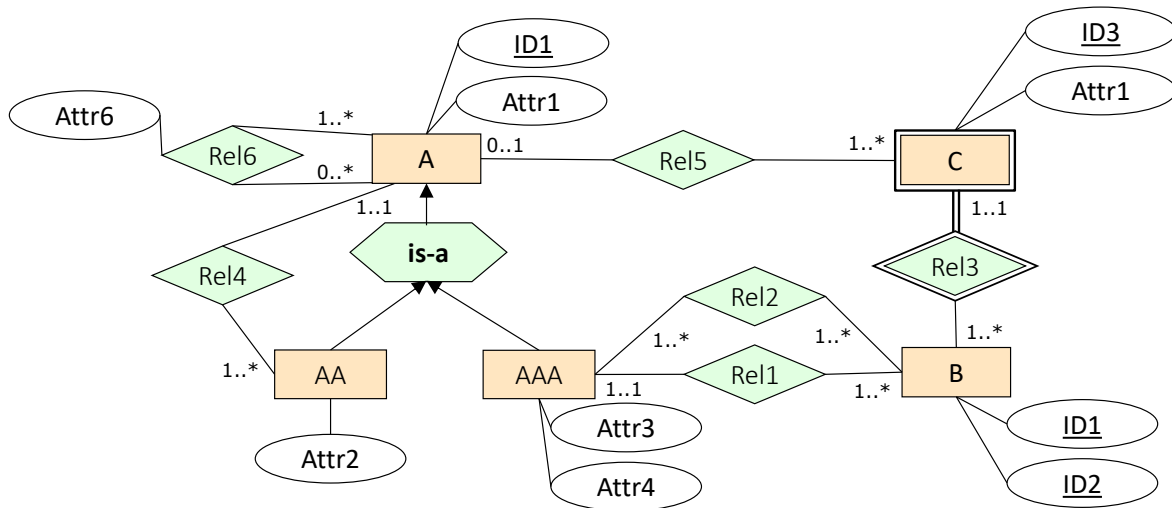


Abbildung 2: Entity Relationship-Diagramm

Kennzeichnen Sie im logischen Schema die jeweiligen Primärschlüssel und Fremdschlüssel sowie Kombinationen davon! Achten Sie auf eine eindeutige Zuordnung zwischen Fremdschlüssel und referenziertem Primärschlüssel und kennzeichnen Sie bei Bedarf die Fremdschlüsselattribute zusätzlich mit dem Namen der Relation des Primärschlüssels. Zum Beispiel X(ID, Attr1) wird referenziert in Y als Y(..., X.ID, ...).

b) Welche Vereinfachungen wären an Ihrem logischen Schema möglich, wenn **Fremdschlüssel** auch den Wert **NULL** annehmen dürfen? (0,5 Punkte)

3. Schulungsdatenbank, Teil 2 (2er-Gruppen wie bei Teil 1)

(6 Punkte)

Modellieren Sie das in der Übung 2 (Aufgabe 4) erstellte Entity Relationship Modell mit Hilfe des Werkzeuges ERwin und berücksichtigen Sie vorhandene Korrekturhinweise der Tutoren. Die Abbildung im Datenmodellierungswerkzeug ERwin **erfordert entsprechende Anpassungen** (bspw. steht Ihnen keine (min,max)-Notation zur Verfügung) – **dokumentieren** Sie diese entsprechend in der Lösung. Many-to-Many Relationen müssen in der Erwin Modellierung nicht aufgelöst werden, sofern sie kein zusätzliches Attribut enthalten. Transformieren Sie anschließend das konzeptuelle bzw. logische Modell in ein **relationales Modell** und wandeln Sie Entitäten und Beziehungen in entsprechende Relationen um. Achten Sie darauf, dass für die Abbildung von Beziehungen keine NULL-Werte entstehen und transformieren Sie auch die Generalisierungen in eine entsprechende relationale Abbildung.

4. Schachturnier (2-er Gruppen)

(8 Punkte)

Für die Verwaltung von Schachturnieren in Gruppen (Teams) soll ein relationales Datenbankschema entworfen werden. Erstellen Sie ein entsprechendes ER-Diagramm in ERwin und wandeln Sie dieses in ein Relationenmodell um, das keine NULL-Werte enthält. Hinweis: Achten Sie bei der Modellierung vor allem auf die korrekte Verwendung von identifizierenden Beziehungen und Rollen.

1. Zu einem Schach-Team ist ein eindeutiger Teamname bekannt, außerdem werden die zum Team (grundsätzlich) zugehörigen Spieler*innen erfasst. Zu jedem Team sind die Spielklasse (z. B.: Landesliga, Staatsliga) und die Team-Adresse bekannt.
2. Jedes Team setzt sich aus mehreren Spieler*innen zusammen. Zu jedem Spieler und jeder Spielerin ist die Spielstärke (R) bekannt. Zusätzlich wird das Eintrittsdatum erfasst, an dem die Spieler*innen in das Team gekommen sind. Spieler*innen werden durch ihre Spielpassnummer identifiziert. Es können auch Spieler*innen in der Datenbank sein, die aktuell zu keinem Team gehören (dann ist auch kein Eintrittsdatum erfasst). Ein*e Spieler*in kann immer nur zu einem Team gehören.
3. Spieler*innen müssen jedoch nicht an allen Turnieren teilnehmen. Die Nominierung der konkreten Spieler*innen für ein Turnier erfolgt erst bei der Auslosung der einzelnen Runden.
4. Jedem Schachturnier wird eine eindeutige Turniernummer zugeordnet. Zu jedem Turnier sind der Austragungsort und der Austragungstermin bekannt. Zu jedem Turnier können sich teilnehmende Teams anmelden. Ein Turnier kann auch an der Adresse eines Teams ausgetragen werden.
5. Bei der Anmeldung zu einem Turnier wird nur das Team und die Anzahl der bei diesem Turnier teilnehmenden Spieler*innen angegeben. Diese Anzahl ist nicht für alle Teams und Turniere gleich.
6. Ein Schachturnier besteht aus mehreren Spiel-Runden, wobei für jedes Schachturnier die Rundenzählung neu beginnt. In jeder Runde werden alle Teams **paarweise** zu Spielen ausgelost, d.h. in einer Runde spielt ein*e Spieler*in eines Teams gegen eine*n Spieler*in eines anderen Teams, sodass von jedem Team pro Runde ein*e Spieler*in spielt. Es ist nicht ausgeschlossen, dass zwei Teams während eines Turniers öfter als einmal gegeneinander spielen.
7. Zu jeder Runde gehören mehrere Partien, die innerhalb der Runden fortlaufend nummeriert werden. Die zwei konkreten Spieler*innen der beiden ausgelosten Teams dieser Partie und deren jeweilig erzielte Punkte (Ergebnis) werden in der Partie gespeichert.

Notation:

- FK := Fremdschlüssel
- PK := Primärschlüssel

1.) Laborverwaltung

Person: {[PersNr:Number, Name:String, Geburtsdatum:Date]}

Administrator: {[PersNr:Number, Gehalt:Number, Arbeitsbeginn:Date]}

Student: {[PersNr:Number, Studiengang:String, Semester:Number]}

benutzt: {[StudentPersNr:Number, LaborId:Number, Nutzungsbeginn:Number, Nutzungsende:Number]} // Fremd- und Primärschlüssel – für die eindeutige Session noch den Nutzungsbeginn

zuständig: {[LaborId:Number, AdminPersNr:Number]} // Note: Diese Tabelle ist nur notwendig, wenn (so wie im Diagramm angegeben) ein Labor keinen oder einen Administrator haben kann. Dann gilt: 2 Fremdschlüsseln, LaborId ist auch Primärschlüssel, weil ein Labor genau einen Admin hat, wenn es einen Eintrag in dieser Tabelle dafür gibt. Wenn gilt, dass ein Labor genau einen Administrator hat (so wie oben in der Beschreibung angegeben), dann ist die Tabelle überflüssig und bei Labor kann die PersNr des zugehörigen Administrators als Fremdschlüssel gespeichert werden.

Labor: {[LaborId:Number, Sitzplätze:Number, Typ:String,]}

Hardware: {[HardwareId:Number, Bezeichnung:String, Kaufpreis:Number, Kaufdatum:Date, LaborId:Number]}

2.) Relationales Modell

// Die Datentypen werden in dieser Bearbeitung weggelassen, weil diese Informationen auch nicht aus dem Diagramm ersichtlich sind.

A: {[ID1, Attr1, AA.A.ID1]} // Rel4: AA.A.ID1

Rel6: {[A.ID1_1, A.ID1_2, Attr6]}

// hier: AA, AAA is-a A umsetzen

AA: {[A.ID1, Attr2]}

AAA: {[A.ID1, Attr3, Attr4, B.ID1, B.ID2]} // Rel1 (AAA-B) erfüllt

Rel5: {[A.ID1, C.ID3, C.B.ID1, C.B.ID2]} // Rel5 umgesetzt: ein A steht genau 1-mal in der Beziehung, wenn die Zeile in der Tabelle existiert (stünde ein A nicht in Beziehung, würde die Tabellenzeile nicht existieren). Deshalb ist nur A.ID1 PK, damit ein bestimmtes A nur einmal vorkommen kann. Alle sind jedoch FKs.

C: {[ID3, B.ID1, B.ID2, Attr1]} // identifizierende Beziehung, Rel3 erfüllt

B: {[ID1, ID2]}

// Rel2: n:m: eigene Tabelle anlegen – PKs von AAA und B

Rel2: {[AAA.A.ID1, B.ID1, B.ID2]} // alles PK und FK – alle Kombinationen sind möglich wegen n:m

b) Vereinfachungen, wenn NULL-Werte erlaubt wären:

Effektive Änderung nur bei Rel5

- Rel1: ohnehin ohne eigene Tabelle
- Rel2: n:m Beziehung – braucht sowieso eine eigene Tabelle
- Rel3: 1:n (genau 1), braucht sowieso keine eigene Tabelle
- Rel4: 1:n (genau 1), braucht sowieso keine eigene Tabelle
- Rel5: bräuchte keine eigene Tabelle, bei A könnte der PK von C als FK eingefügt werden. Wenn A 0-mal in der Beziehung mit C steht, dann wäre dieser FK einfach null.
- Rel6: n:m Beziehung mit Attribut – braucht sowieso eigene Tabelle

3.) Gruppenarbeit mit Julian Bichler

KAngebot: {[AngebotID:Number]}

istTeilVon: {[KAngebot.AngebotID:Number, Kurs.KursID:Number]}

Kurs: {[KursID:Number, Thema:String, Zielgruppe:String, Schwierigkeit:String, Dauer:Number]}

kannHalten: {[Kurs.KursID:Number, Dozent.DozentID:Number]}

Dozent: {[DozentID:Number, Bankverbindung:String, Vorname:String, Nachname:String]}

hältLVA: {[Dozent.DozentID:Number, LVA.LV-ID:Number, LVA.Kurs.KursID:Number, Honorar:Number]}

LVA: {[LV-ID:Number, Beginn:Number, Datum:Date, Ort:String, Kurs.KursID:Number]}

OffeneLVA: {[LVA.LV-ID:Number, LVA.Kurs.KursID]}

nimmtTeil: {[Einzelperson.Kunde.KundenID:Number, OffeneLVA.LVA.LV-ID:Number, OffeneLVA.LVA.Kurs.KursID]}

GeschlLVA: {[LVA.LV-ID:Number, LVA.Kurs.KursID:Number, Firma.Kunde.KundenID:Number]}

Kunde: {[KundenID:Number, Name:String, Bankverbindung:String, Anschrift:String]}

Einzelperson: {[Kunde.KundenID:Number]} // is-a Kunde

Firma: {[Kunde.KundenID:Number]} // is-a Kunde

